

Interface Specification

Line Buffer & MAC Pipeline — 2D Convolution FPGA Accelerator

Module-Level Port Definitions for RTL Implementation

Baseline parameters: 16×16 image • 8-bit grayscale pixels • 3×3 kernel • valid padding, stride 1

Target: mid-range Xilinx FPGA (DSP48-based multiply-accumulate)

Version 0.1 — Draft for team review

Scope: Line Buffers → Window Generator → Kernel Register Bank → Multiplier Array → Adder Tree → Output Assembly →
Output Interface → Frame Buffer → VGA Controller

1. Overview

This document defines the port-level interface for each RTL module in the line buffer, MAC, and display pipeline. Modules connect in a single streaming chain from pixel input through to VGA display output, with the Kernel Register Bank supplying constant coefficients to the Multiplier Array via a side path. Coordinates (row/col) are carried alongside the data through every compute stage as a shadow pipeline so the Output Assembly block always knows which output pixel it is producing. Downstream of the compute pipeline, the Output Interface writes results into a dual-port Frame Buffer, which the VGA Controller reads independently to drive a live display.

Global Parameters

Dir	Signal	Width	Description
—	PIXEL_WIDTH	8	Unsigned grayscale pixel width
—	COEF_WIDTH	8	Signed kernel coefficient width
—	IMG_SIZE	16	Baseline image dimension (16×16)
—	PRODUCT_WIDTH	16	Signed product width (pixel × coefficient)
—	SUM_WIDTH	20	Signed accumulator width (sum of 9 products)

2. Input Interface — pixel_rx_if

Accepts one pixel per cycle from the PC-facing stream and generates row/column position tracking used by every downstream stage.

Dir	Signal	Width	Description
in	clk	1	System clock
in	rst_n	1	Active-low synchronous reset
in	pixel_in	8	Current pixel value from streaming source
in	pixel_valid_in	1	Asserted when pixel_in is valid this cycle
in	frame_start	1	Pulses at the start of a new 16×16 frame
out	pixel_out	8	Forwarded pixel value
out	pixel_valid_out	1	Forwarded valid strobe
out	row_cnt	4	Current pixel row index (0–15)
out	col_cnt	4	Current pixel column index (0–15)

3. Line Buffers — line_buffer_fifo

Two SRL-based FIFOs (depth 16, width 8) store the previous two image rows, providing the vertical pixel reuse needed to build a 3×3 window each cycle.

Dir	Signal	Width	Description
in	clk	1	System clock
in	rst_n	1	Active-low synchronous reset
in	pixel_in	8	Incoming pixel (current row)
in	valid_in	1	Valid strobe for pixel_in
out	row_top	8	Pixel from two rows ago
out	row_mid	8	Pixel from one row ago
out	row_bot	8	Current-row pixel (pass-through)
out	valid_out	1	High once both FIFOs hold enough data

4. Window Generator — window_gen

Assembles the 3×3 pixel neighborhood using horizontal shift registers fed by the three row taps, and masks output validity for border positions under valid padding.

Dir	Signal	Width	Description
in	clk	1	System clock
in	rst_n	1	Active-low synchronous reset
in	row_top	8	Top row tap from line buffers
in	row_mid	8	Middle row tap from line buffers
in	row_bot	8	Bottom row tap from line buffers
in	valid_in	1	Valid strobe from line buffers

Dir	Signal	Width	Description
in	row_cnt	4	Row index passthrough from input interface
in	col_cnt	4	Column index passthrough from input interface
out	win[0:8]	8 each	9 window pixels (3×3 neighborhood)
out	win_valid	1	High only for a full, non-border window
out	out_row	4	Output pixel row coordinate
out	out_col	4	Output pixel column coordinate

5. Kernel Register Bank — kernel_regs

Configuration-side block, separate from the streaming datapath. Loads the 9 signed coefficients once before streaming begins and holds them constant for the duration of the frame.

Dir	Signal	Width	Description
in	clk	1	System clock
in	rst_n	1	Active-low synchronous reset
in	kernel_load	1	Write-enable pulse for coefficient load
in	kernel_addr	4	Coefficient index (0–8)
in	kernel_data	8, signed	Coefficient value to write
out	k[0:8]	8 each, signed	Constant coefficient bus to multiplier array

6. Multiplier Array — mac_mult_array

9 parallel DSP48-inferred multipliers, each multiplying one window pixel by its corresponding kernel coefficient. Output is registered (1 pipeline stage).

Dir	Signal	Width	Description
in	clk	1	System clock
in	rst_n	1	Active-low synchronous reset
in	win[0:8]	8 each	9 window pixels from window generator
in	k[0:8]	8 each, signed	9 kernel coefficients from kernel bank
in	valid_in	1	= win_valid
in	out_row	4	Coordinate passthrough (input)
in	out_col	4	Coordinate passthrough (input)
out	prod[0:8]	16 each, signed	9 registered pixel × coefficient products
out	valid_out	1	valid_in delayed by 1 cycle
out	out_row_d	4	Delayed coordinate passthrough
out	out_col_d	4	Delayed coordinate passthrough

7. Adder Tree — adder_tree

Balanced reduction of the 9 products to a single sum (9→5→3→2→1), with a pipeline register between each level for timing closure.

Dir	Signal	Width	Description
in	clk	1	System clock
in	rst_n	1	Active-low synchronous reset
in	prod[0:8]	16 each	9 products from multiplier array
in	valid_in	1	Valid strobe from multiplier array
in	out_row_d	4	Coordinate passthrough (input)
in	out_col_d	4	Coordinate passthrough (input)
out	conv_sum	20, signed	Final convolution result for one output pixel
out	valid_out	1	Delayed by number of tree pipeline stages
out	out_row_f	4	Fully delayed coordinate, aligned with conv_sum
out	out_col_f	4	Fully delayed coordinate, aligned with conv_sum

8. Output Assembly — output_assembly

Final pipeline register stage; optionally clips/saturates the result and hands off to memory write or an output stream, decoupling the compute pipeline from the output interface.

Dir	Signal	Width	Description
in	clk	1	System clock
in	rst_n	1	Active-low synchronous reset
in	conv_sum	20, signed	Convolution result from adder tree
in	valid_in	1	Valid strobe from adder tree
in	row_f	4	Output row coordinate
in	col_f	4	Output column coordinate
out	pixel_out	8 or 20	Final output pixel (clipped/full-width)
out	pixel_out_valid	1	Output valid strobe
out	mem_wr_en	1	Memory write enable (if writing to BRAM)
out	mem_wr_addr	8	Write address = $\text{row_f} \times 14 + \text{col_f}$

9. Output Interface — output_if

Bridges the compute pipeline to the frame buffer memory. Takes the final pixel and write-strobe signals from Output Assembly and drives the write port of the dual-port BRAM, including its write clock domain.

Dir	Signal	Width	Description
in	clk	1	System clock
in	rst_n	1	Active-low synchronous reset
in	pixel_out	8 or 20	Final output pixel from output_assembly
in	pixel_out_valid	1	Valid strobe from output_assembly
in	mem_wr_en	1	Memory write enable from output_assembly
in	mem_wr_addr	8	Memory write address from output_assembly
out	write_data	8	Pixel data driven to frame buffer Port A
out	write_en	1	Write enable driven to frame buffer Port A
out	write_addr	8	Write address driven to frame buffer Port A
out	wr_clk	1	Write clock supplied to frame buffer Port A
out	rd_clk	1	Read clock supplied to frame buffer Port B

10. Frame Buffer — dual_port_bram

Dual-port BRAM storing one full convolved frame. Port A is write-only, driven by the Output Interface. Port B is read-only, driven by the VGA Controller, allowing simultaneous write and display readout.

Dir	Signal	Width	Description
in	wr_clk	1	Port A write clock
in	write_en	1	Port A write enable
in	write_addr	8	Port A write address
in	write_data	8	Port A write data (pixel value)
in	rd_clk	1	Port B read clock
in	rd_addr	8	Port B read address, from VGA controller
out	rd_data	8	Port B read data, to VGA controller

11. VGA Controller — vga_ctrl

Reads the stored frame from the frame buffer over Port B and generates standard VGA timing and color output for display, including horizontal/vertical sync and blanking.

Dir	Signal	Width	Description
in	vga_clk	1	Pixel clock for VGA timing
in	vga_rst_n	1	Active-low synchronous reset
in	rd_data	8	Pixel data read back from frame buffer
out	rd_addr	8	Read address driven to frame buffer Port B
out	VGA_R	8	Red color channel output

Dir	Signal	Width	Description
out	VGA_G	8	Green color channel output
out	VGA_B	8	Blue color channel output
out	VGA_HS	1	Horizontal sync output
out	VGA_VS	1	Vertical sync output
out	VGA_BLANK_N	1	Active-low blanking output
out	VGA_SYNC_N	1	Active-low composite sync output
out	VGA_CLK	1	VGA pixel clock output

12. Integration Notes

- Handshake protocol is valid-only (no ready/backpressure) — the source rate is fixed and known, which keeps every module's control logic simple.
- Coordinate passthrough (row_cnt / col_cnt and its delayed forms) must be carried alongside the data through every pipeline stage so Output Assembly always writes to the correct address, regardless of how many pipeline registers are added or removed while tuning timing.
- The frame buffer is dual-clock (wr_clk / rd_clk) — the write side runs on the pipeline's system clock while the read side runs on vga_clk, so address/data crossing between the two domains relies on the BRAM's native dual-port behavior rather than a manual CDC synchronizer.
- Each module should be verified standalone against known input/output sequences before integration.